FPT UNIVERSITY

HO CHI MINH CITY CAMPUS

EMBEDDED LINUX SYSTEMS | LOS201

ASSIGNMENT 01

DESIGN, IMPLEMENTATION, AND VERIFICATION
OF LINUX DEVICE DRIVERS ON ARMv7

Ioctl character driver • Misc counter device • QEMU vexpress-a9

TEAM MEMBERS (04)

No.	Full name	Student ID
01	Lê Tiến Thành	SE201799
02	Lê Tấn Lộc	SE201412
03	Trần Đình Khôi Nguyên	SE200558
04	Trần Trương Tuấn Vũ	SE196467

Lecturer: Phạm Thế Vinh

Experimental platform: Linux 5.15.0 • ARMv7 Cortex-A9 • QEMU vexpress-a9

Ho Chi Minh City, October 2026

OVERVIEW

REPORT SUMMARY AND SCOPE

This report presents two kernel modules with different registration models that both rely on the Linux I/O subsystem. Task 1 extends a character driver with four ioctl commands to control the buffer and retrieve internal status. Task 2 builds an auto-increment misc device, allows the counter to be reset through write operations, and publishes runtime statistics through procfs. Both modules are cross-compiled for ARMv7 and executed on Linux 5.15.0 in QEMU vexpress-a9.

Instead of describing the work as a simple sequence of commands, the report follows a technical narrative: requirements → architecture → internal mechanisms → evidence → comparison and evaluation → theoretical foundation. The diagrams were redrawn from the implemented architecture, while every PASS count, major/minor number, and statistics block was taken from the actual terminal output and log files of the project.

Measured result summary

Item	Measured value	Meaning
Task 1 - ioctl app	18 PASS / 0 FAIL	4 commands, 2 modes, RESET, errors, and stats
Task 1 - script	5 PASS / 0 FAIL	Major registration, device node, unload
Task 2 - counter script	8 PASS / 0 FAIL	read/write, procfs, invalid input, cleanup
Target environment	ARMv7 • Linux 5.15.0	Module vermagic matches the kernel
Registered devices	241:0 and 10:125	Character device and misc device

Content roadmap

01. Environment and build workflow → 02. Overall system architecture → 03. Ioctl analysis → 04. Misc counter design → 05. Comparison of three driver families and synchronization choices → 06. Theoretical questions → 07. Validation and conclusion.

Data cross-checking convention

- Primary functional reference: the official assignment specification ASSIGNMENT01_EmbeddedLinux.docx; the requirements for Task 3 and Task 4 are preserved exactly.
- Experimental evidence sources: test_asgn1_output.txt, test_counter_output.txt, dmesg_full.txt, build_task1.txt, and build_task2.txt.

BUILD ENVIRONMENT AND MODULE VERIFICATION

Component	Verified configuration
Host	Ubuntu Linux; ARM cross-toolchain arm-linux-gnueabihf-gcc
Target	QEMU vexpress-a9, Cortex-A9, 2 vCPU, RAM 512 MiB
Kernel	Linux 5.15.0; CONFIG_MODULES=y
Task 1 module	asgn1_driver.ko, ARM ELF 32-bit EABI5
Task 2 module	counter_driver.ko, ARM ELF 32-bit EABI5
Vermagic (both)	5.15.0 SMP mod_unload ARMv7 p2v8
Test application	test_asgn1_app: ARM ELF 32-bit, static

Out-of-tree compilation is performed through Kbuild with ARCH=arm and CROSS_COMPILE=arm-linux-gnueabihf-. The build logs show the sequence CC [M] → MODPOST → LD [M] with no undefined-symbol errors; this proves that the modules pass the link stage, while only runtime testing can confirm correct behavior.

```
===== ASSIGNMENT 01 - BUILD VERIFICATION =====
===== TASK 1 =====
        modules
make[1]: Entering directory '/home/letienthanh/embedded_lab1/kernel/linux-5.15'
  CC [M]  /home/letienthanh/LOS201/embedded-linux-assignment01/task1_ioctl/asgn1_driver.o
  MODPOST /home/letienthanh/LOS201/embedded-linux-assignment01/task1_ioctl/Module.symvers
  CC [M]  /home/letienthanh/LOS201/embedded-linux-assignment01/task1_ioctl/asgn1_driver.mod.
o
  LD [M]  /home/letienthanh/LOS201/embedded-linux-assignment01/task1_ioctl/asgn1_driver.ko
make[1]: Leaving directory '/home/letienthanh/embedded_lab1/kernel/linux-5.15'
===== TASK 2 =====
        modules
make[1]: Entering directory '/home/letienthanh/embedded_lab1/kernel/linux-5.15'
  CC [M]  /home/letienthanh/LOS201/embedded-linux-assignment01/task2_misc/counter_driver.o
  MODPOST /home/letienthanh/LOS201/embedded-linux-assignment01/task2_misc/Module.symvers
  CC [M]  /home/letienthanh/LOS201/embedded-linux-assignment01/task2_misc/counter_driver.mod
.o
  LD [M]  /home/letienthanh/LOS201/embedded-linux-assignment01/task2_misc/counter_driver.ko
make[1]: Leaving directory '/home/letienthanh/embedded_lab1/kernel/linux-5.15'
===== ARM ARCHITECTURE =====
task1_ioctl/asgn1_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildI
D[sha1]=8b205cb26e8b5b97f5684e533e20776af9d9c26f, with debug_info, not stripped
task2_misc/counter_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), Build
ID[sha1]=ea125c46dab74368339c6b368209e9c048f58e87, with debug_info, not stripped
===== KERNEL COMPATIBILITY =====
5.15.0 SMP mod_unload ARMv7 p2v8
5.15.0 SMP mod_unload ARMv7 p2v8
===== USERSPACE TEST APP =====
task1_ioctl/test_asgn1_app: ELF 32-bit LSB executable, ARM, EABI5 version 1 (GNU/Linux), sta
tically linked, BuildID[sha1]=a29fff48b93b30f48d4504be353e3b83edaa5349, for GNU/Linux 3.2.0,
 not stripped
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/LOS201/embedded-linux-assignment01$
```

Figure 1. Evidence of a clean build, ARM EABI5 target format, and matching vermagic.

OVERALL ARCHITECTURE OF THE TWO DRIVERS

The VFS routes operations associated with each file descriptor to the appropriate file_operations instance. The orange branch represents data returning from the driver to userspace through copy_to_user(); the procfs branch is separated from the device path, uses seq_file, and reads the Task 2 DATA LAYER directly instead of invoking read() on /dev/counter. Sysfs only exposes the device object and does not replace the data path.

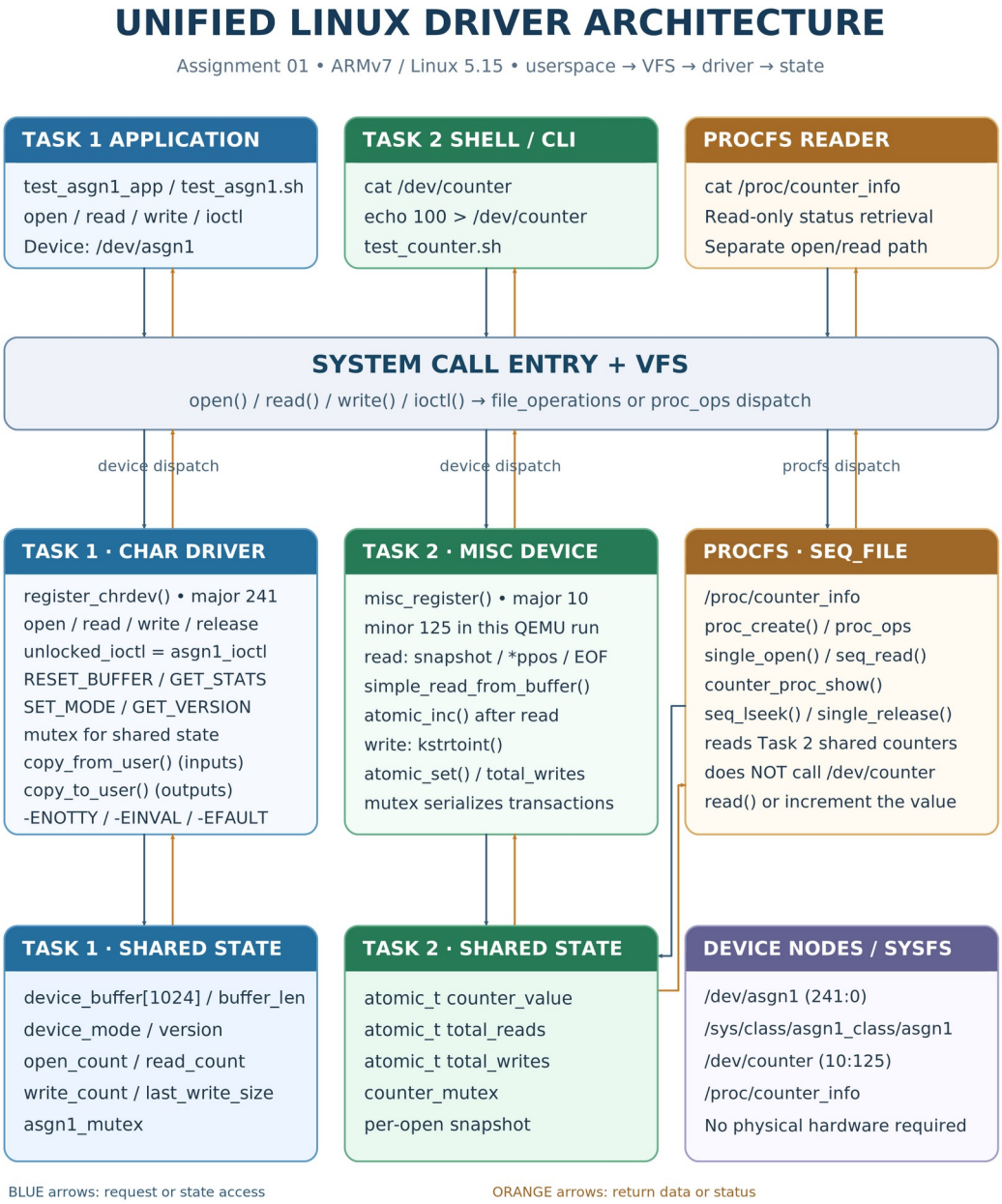


Figure 2. Consolidated architecture and bidirectional data flow: ioctl()/read() return to userspace through uaccess, while /proc/counter_info follows its own VFS → seq_file path.

03 | TASK 1

DISSECTING THE IOCTL FLOW AND THE FOUR CONTROL COMMANDS

When ioctl(fd, cmd, arg) is called, the VFS enters file_operations.unlocked_ioctl of asgn1_driver. The driver decodes the command and determines whether data must cross the user/kernel boundary. The ioctl control flow shows all four handling paths, the returned error codes, and the point where the mutex protects shared state.

Ioctl control flow: The test application opens /dev/asgn1 and passes the command and argument through ioctl() and the VFS to asgn1_ioctl(). The RESET_BUFFER branch clears the buffer and the resettable statistics under asgn1_mutex. GET_STATS returns the five-field snapshot through copy_to_user(); SET_MODE receives an integer through copy_from_user() and rejects values other than 0 or 1; GET_VERSION returns a bounded version string. Unknown commands return -ENOTTY, invalid modes return -EINVAL, and an unsuccessful user-memory copy returns -EFAULT.

03 | TASK 1

TASK 1 — STATE DESIGN AND EXTERNAL INTERFACE

1. State model of asgn1_driver

The driver maintains an internal buffer together with its stored length, the current write-processing mode, a version string, and five statistics fields. The asgn1_mutex mutex protects operations that can read or modify shared state; this prevents concurrent processes from overwriting the buffer or corrupting the statistics. open_count records the total number of open events, while read_count and write_count reflect the number of successful read/write operations according to the driver logic.

Command	Data crossing the boundary	Verified effect
RESET_BUFFER	_IO, no argument	Clears data; read/write_count=0; buffer_len=0
GET_STATS	_IOR, struct with 5 int = 20 B	Returns open/read/write, len, last_write_size
SET_MODE	_IOW, int 4 B	0: prepend "[KERNEL]"; 1: Echo; other: -EINVAL
GET_VERSION	_IOR, char[64]	Returns "asgn1_driver v1.0", limited to 64 B

2. Example macros from the real header

```
#define ASGN1_IOC_MAGIC      'A'
#define ASGN1_VERSION_LEN    64
#define ASGN1_RESET_BUFFER   _IO(ASGN1_IOC_MAGIC, 1)
#define ASGN1_GET_STATS      _IOR(ASGN1_IOC_MAGIC, 2, struct asgn1_stats)
#define ASGN1_SET_MODE       _IOW(ASGN1_IOC_MAGIC, 3, int)
#define ASGN1_GET_VERSION    _IOR(ASGN1_IOC_MAGIC, 4,
char[ASGN1_VERSION_LEN])
```

3. Difference between input payload and stored data

In Echo mode, write("hello", 5) stores five bytes and a later read returns exactly “hello”. In mode 0, write("world", 5) produces “[KERNEL]world”: buffer_len becomes 13 bytes, while last_write_size remains 5. This accounting distinguishes the size of the userspace input from the amount of data actually stored in the kernel buffer.

> TECHNICAL NOTE During the test sequence, RESET_BUFFER does not clear open_count; that field increases from 1 to 5 across successive opens and represents the module activity history rather than the current buffer contents.

03 | TESTING / REGISTRATION

TASK 1 — /dev/asgn1 REGISTRATION, SYSFS, AND MAJOR RELEASE

With major 241, insmod registers the character driver asgn1 and creates the object /sys/class/asgn1_class/asgn1. The screenshot confirms lsmod, /proc/devices, and the device node /dev/asgn1 (241:0); after the official test run, dmesg shows “driver unloaded” and /proc/devices no longer lists the registered major number. The presence of a /dev node alone is not proof that the module is still loaded.

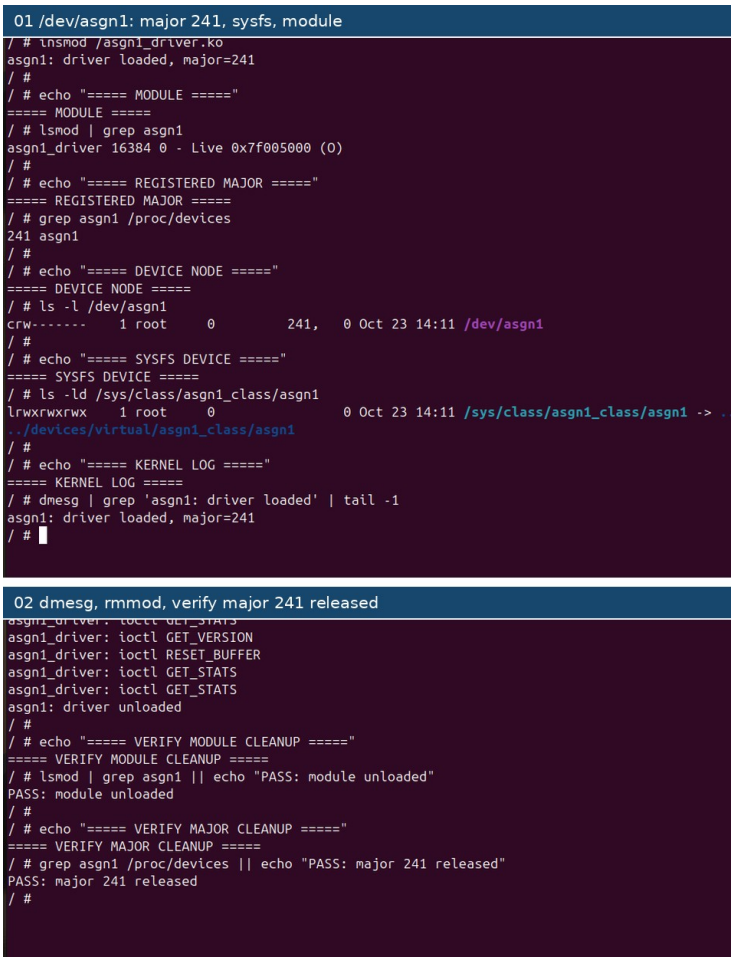


Figure 3. Character-device registration at major 241, sysfs link, insmod log, and QEMU evidence of rmmod/cleanup.

Snippet 1 — switch(cmd) flow inside asgn1_ioctl() (taken from the real source; /* ... */ marks lines omitted only to keep the report compact; the full implementation remains in asgn1_driver.c).

```
switch (cmd) {
case ASGN1_RESET_BUFFER:
    mutex_lock(&asgn1_mutex);
    memset(device_buffer, 0, BUFFER_SIZE);
    buffer_len = 0;
    read_count = 0; write_count = 0;
    mutex_unlock(&asgn1_mutex);
    /* ... remaining lines omitted ... */
case ASGN1_GET_STATS:
    /* ... populate stats under mutex ... */
    if (copy_to_user((void __user *)arg,
                     &stats, sizeof(stats)))
        return -EFAULT;
case ASGN1_SET_MODE:
    if (copy_from_user(&new_mode,
                       (int __user *)arg, sizeof(new_mode)))
        return -EFAULT;
    /* ... GET_VERSION branch ... */
default:
    pr_warn("asgn1_driver: unsupported ioctl 0x%x\n", cmd);
    return -ENOTTY;
}
```


TASK 1 — IOCTL CODES, THE TWO MODES, AND VERSION

The ARM test program recognizes all four standard commands defined in the header. Echo mode returns “hello” as the same five bytes; Kernel mode prepends “[KERNEL]” when “world” is written and later returns “[KERNEL]world” (13 bytes). GET_VERSION returns “asgn1_driver v1.0”; the invalid-command case is separated in Figure 6.

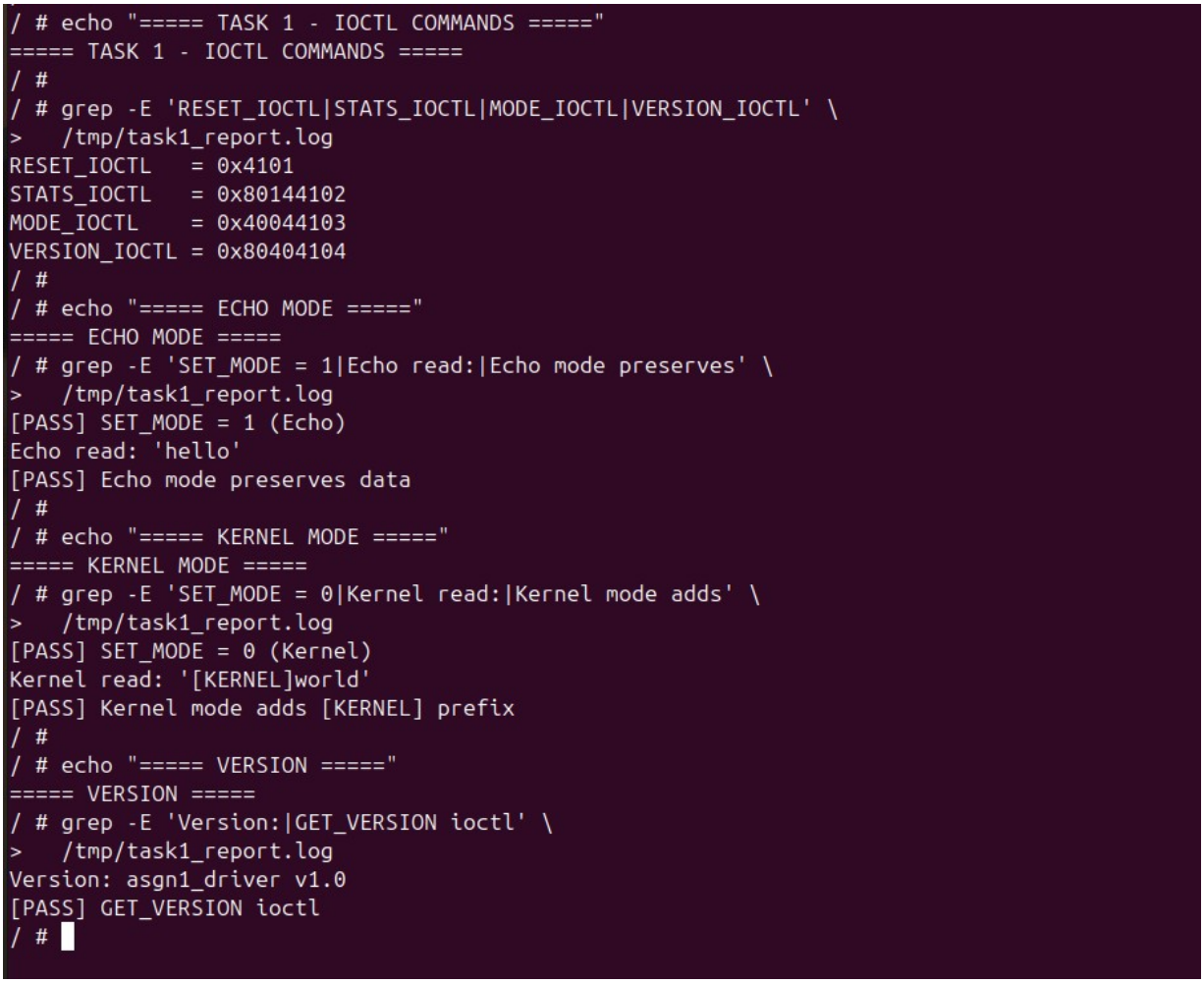


Figure 4. The four ioctl codes (0x4101, 0x80144102, 0x40044103, 0x80404104), the two data modes, and GET_VERSION.

03 | TESTING / STATISTICS

TASK 1 — GET_STATS AT THREE STATE MILESTONES

GET_STATS returns the five fields of struct asgn1_stats. In Echo mode, buffer_len=5; in Kernel mode, buffer_len=13 because of the 8-byte prefix. After the final RESET, write_count, read_count, buffer_len, and last_write_size return to 0, while open_count=5 is intentionally preserved by RESET_BUFFER. The figure contains the three sections STATS AFTER ECHO, STATS AFTER KERNEL MODE, and FINAL STATS directly from the test run.

Field	After Echo	After Kernel mode	After final RESET
open_count	3	4	5
write_count	1	2	0
read_count	1	2	0
buffer_len	5	13	0
last_write_size	5	5	0

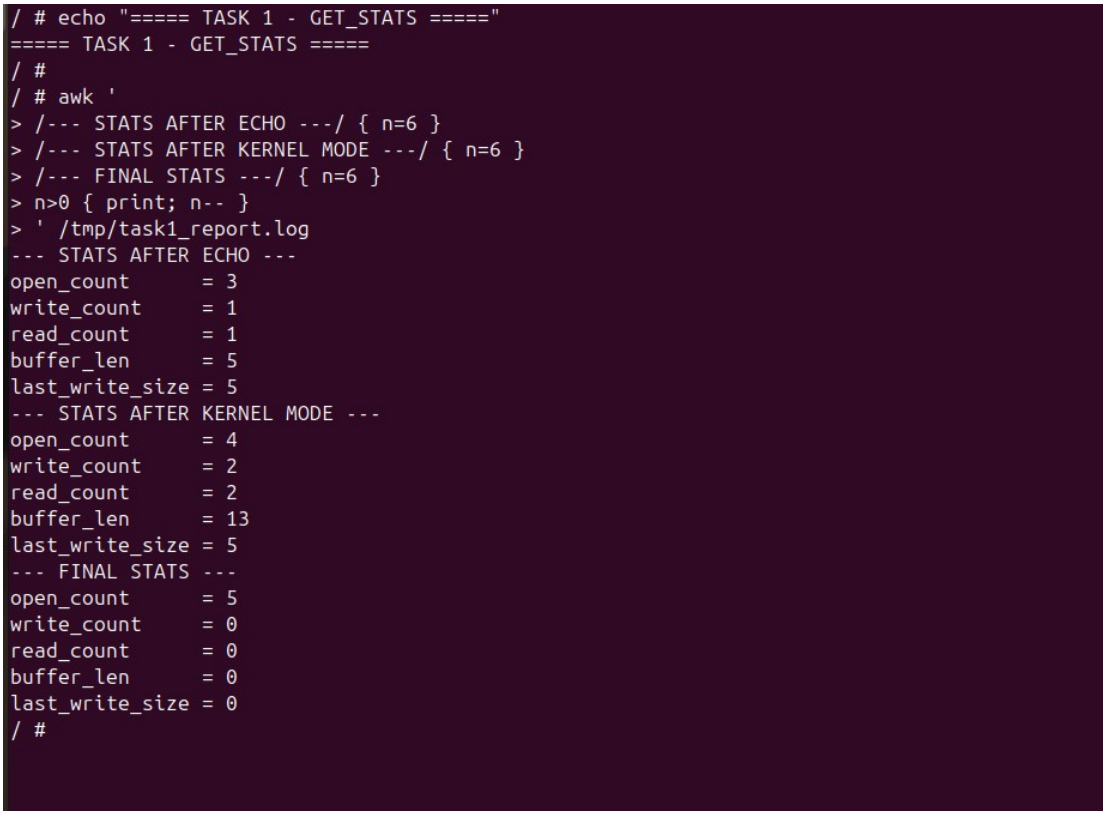


Figure 5. GET_STATS at three moments; all resettable fields return to 0, while open_count remains intact.

During the test sequence, open_count represents how many opens occurred since the module was loaded, not how many records remain in the buffer. Preserving open_count across RESET is consistent with the command definition in the official assignment.

03 | TESTING / ERRORS AND CLEANUP

TASK 1 — BOUNDARY TESTS AND SCRIPT RESULTS

The ioctl value 0xdeadbeef is rejected with -ENOTTY; mode=2 is rejected with -EINVAL. After RESET_BUFFER, read() returns EOF and no data remain. The C program finishes with PASSED: 18, FAILED: 0, while test_asgn1.sh confirms 5 checks at the script level with FAILED: 0. The module is also removed successfully and major 241 is released.

03 RESET, ENOTTY, EINVAL, test 18/18 + 5/5

```
/ #
/ # grep -E '\[PASS\] (RESET_BUFFER|RESET clears|Read after RESET|Invalid ioctl|
Invalid mode|Final RESET_BUFFER|Final counters|Final buffer|GET_VERSION)' \
>   /tmp/task1_report.log
[PASS] RESET_BUFFER ioctl
[PASS] RESET clears buffer and counters
[PASS] Read after RESET returns EOF
[PASS] GET_VERSION ioctl
[PASS] Invalid ioctl returns ENOTTY
[PASS] Invalid mode returns EINVAL
[PASS] Final RESET_BUFFER
[PASS] Final counters are zero
[PASS] Final buffer is empty
/ #
/ # echo "===== COMPLETE TEST SUMMARY ====="
===== COMPLETE TEST SUMMARY =====
/ # grep -E 'PASSED:|FAILED:|\[PASS\] All ioctl test cases' \
>   /tmp/task1_report.log
PASSED: 18
FAILED: 0
[PASS] All ioctl test cases
PASSED: 5
FAILED: 0
/ #
/ # echo "===== RESULT ====="
===== RESULT =====
```

04 kernel log and release confirmation

```
asgn1_driver: ioctl GET_STATS
asgn1_driver: ioctl GET_STATS
asgn1: driver unloaded
/ #
/ # echo "===== VERIFY MODULE CLEANUP ====="
===== VERIFY MODULE CLEANUP =====
/ # lsmod | grep asgn1 || echo "PASS: module unloaded"
PASS: module unloaded
/ #
/ # echo "===== VERIFY MAJOR CLEANUP ====="
===== VERIFY MAJOR CLEANUP =====
/ # grep asgn1 /proc/devices || echo "PASS: major 241 released"
PASS: major 241 released
/ #
```

Figure 6. Evidence of rejecting invalid ioctl/mode values, empty reads after RESET, the 18 + 5 PASS summary, and module cleanup.

The dmesg messages “unsupported ioctl” and “invalid mode” are deliberate negative tests, not system faults. The exit code of the official run is verified by TASK1_EXIT_CODE=0 in the log.

04 | TASK 2

COMPLETE VIEW OF THE MISC COUNTER DEVICE

Unlike Task 1, counter_driver does not need a dedicated major-number range. misc_register() places the device under shared major 10 and selects a dynamic minor value. The read()/write() data path goes through file_operations, while /proc/counter_info uses seq_file to report state without modifying the counter.

Counter-device read/write and procfs paths: misc_register() registers /dev/counter under shared major 10. Each open allocates private state. The first successful read formats a decimal snapshot, copies it through simple_read_from_buffer(), advances *ppos, and increments the counter; subsequent reads on the same descriptor reach EOF. The write path uses copy_from_user() and kstrtoint() to validate a nonnegative ASCII integer before setting the counter. Independently, the /proc/counter_info seq_file handler reads the shared counters without incrementing them.

TASK 2 — COUNTER DESIGN AND FILE-OFFSET SEMANTICS

1. Read does not mean infinite output

Each open of /dev/counter allocates a private_data structure that stores the returned text and an initialized flag. The first read takes a snapshot of value N, formats "N\n", calls simple_read_from_buffer(), and increments counter_value and total_reads only when bytes are actually delivered to userspace. A repeated read on the same file descriptor sees EOF after the offset reaches the end; a new cat command opens a new file descriptor and obtains the next value.

2. Write accepts only a valid ASCII integer

The write path copies text into a bounded kernel buffer, uses kstrtoint() to parse the integer, rejects negative values, and then calls atomic_set() to update the counter. “100” sets the value to 100, “0” restores it to 0, while “-1” and “abc” return -EINVAL. total_writes is incremented only after a valid write.

3. Consistency on multiple CPUs

atomic_t ensures that each atomic_inc(), atomic_set(), or atomic_read() operation has proper atomic semantics. However, the pair “read N then increment N” consists of two distinct steps, so the driver also uses a mutex to serialize the entire transaction together with the statistics update. As a result, the snapshot returned to userspace and the auto-increment event are not interleaved by concurrent reads on different CPUs.

4. Resource and metadata points

Component	Implementation
/dev/counter	miscdevice, major 10, dynamic minor; mode 0666 in the test build
/proc/counter_info	seq_file: counter_value, total_reads, total_writes
Memory	kzalloc private_data in open(); kfree in release()
Module exit	proc_remove() before misc_deregister()
Module metadata	LICENSE GPL, AUTHOR Le Tien Thanh - SE201799, DESCRIPTION

TECHNICAL NOTE The current 0666 permission simplifies automated testing in the emulated environment; on a real device, write permission should normally be restricted to an appropriate user or group.

TASK 2 — DIRECT PROCFS MEASUREMENTS BEFORE WRITE, AFTER WRITE, AND AFTER RESET

test_counter.sh calls cat /proc/counter_info at three points: before write (0/0/0), immediately after write(100) (100/2/1), and immediately after write(0) (0/3/2). Two separate opens of /dev/counter then return 0 and 1. Figure 7 combines the original terminal regions, each labeled with its own measurement point.

Measurement point	counter_value	total_reads	total_writes
Initial	0	0	0
Immediately after write 100	100	2	1
Immediately after write 0	0	3	2

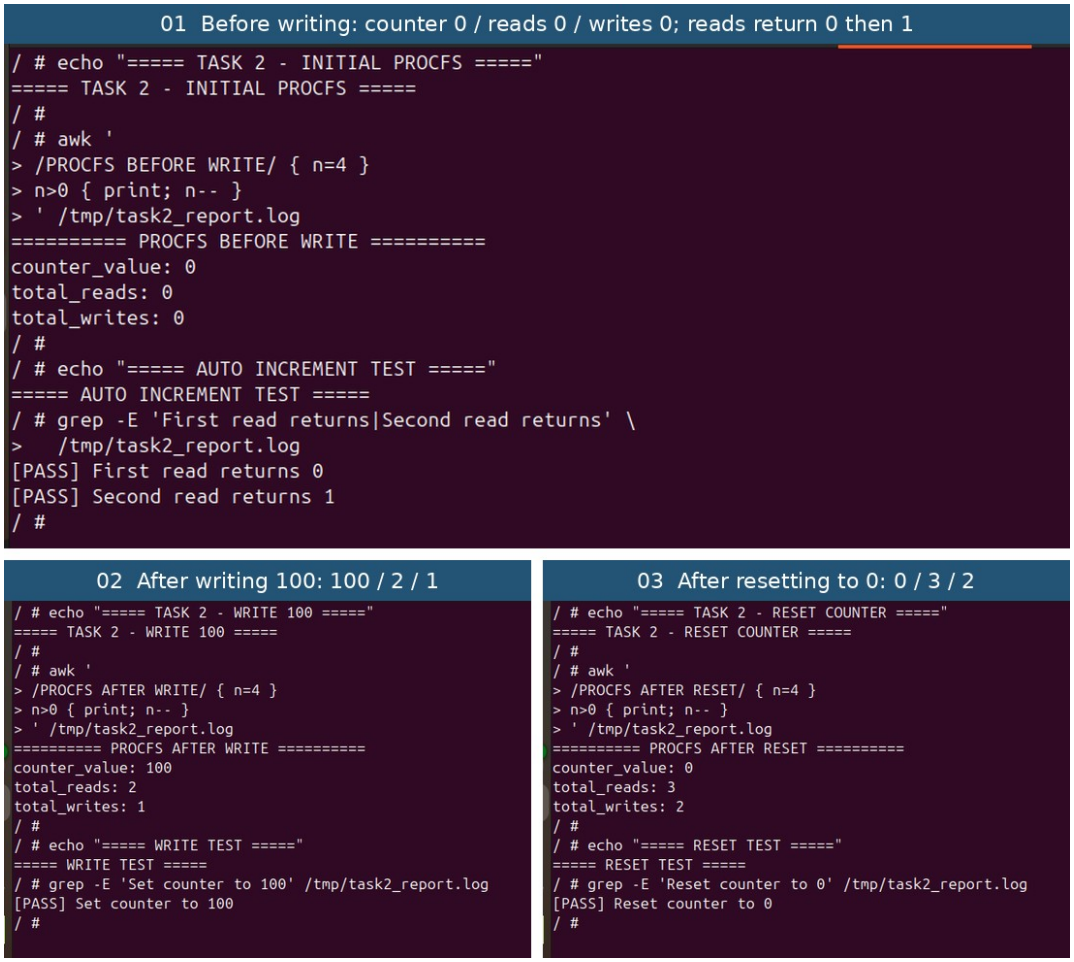


Figure 7. The three actual cat /proc/counter_info snapshots (captured inside the script) together with the 0 → 1 auto-increment test.

After each validation read performed by the script, the counter increases once more; therefore the “FINAL PROCFS” block shows counter_value=1, total_reads=4, and total_writes=2. These values are not inconsistent with the earlier “after reset” checkpoint of 0 shown in the table.

Snippet 2 — miscdevice registration, *ppos handling, atomic_inc, and kstrtoint from counter_driver.c. The selected fragments follow source order but do not form one complete C function.

```
static struct miscdevice counter_misc = {
    .minor = MISC_DYNAMIC_MINOR,
    .name  = DEVICE_NAME,
    .fops  = &counter_fops,
    .mode  = 0666,
};
/* counter_read(): selected lines */
ret = simple_read_from_buffer(buf, count, ppos,
                              data->text, data->length);
if (ret > 0 && !data->initialized) {
    atomic_inc(&counter_value);
    atomic_inc(&total_reads);
    data->initialized = true;
}
/* counter_write(): selected lines */
ret = kstrtoint(input, 10, &value);
if (ret || value < 0)
    return -EINVAL;
```


TASK 2 — REJECTING INVALID INPUT, UNLOAD, AND KERNEL LOG TRACE

In the QEMU run, the kernel assigns major 10 and minor 125. The script rejects “-1” and “abc”, then executes rmmod counter_driver and confirms that /proc/counter_info has been removed. The dmesg trace records loaded, read value, set counter=100, set counter=0, and unloaded. Script result: 8 PASS, 0 FAIL, TASK2_EXIT_CODE=0.

```
/ # echo "===== TASK 2 - UNLOAD ====="
===== TASK 2 - UNLOAD =====
/ #
/ # grep -E 'rmmod counter_driver|procfs cleanup' \
>   /tmp/task2_report.log
[PASS] rmmod counter_driver
[PASS] procfs cleanup
/ #
/ # echo "===== MODULE STATUS ====="
===== MODULE STATUS =====
/ # lsmod | grep counter_driver || echo "PASS: module unloaded"
PASS: module unloaded
/ #
/ # echo "===== PROCFS CLEANUP ====="
===== PROCFS CLEANUP =====
/ # if [ ! -e /proc/counter_info ]; then
>     echo "PASS: /proc/counter_info removed"
> else
>     echo "FAIL: /proc/counter_info still exists"
> fi
PASS: /proc/counter_info removed
/ #
/ # echo "===== KERNEL LOG ====="
===== KERNEL LOG =====
/ # dmesg | grep counter_driver | tail -12
counter_driver: loaded major=10 minor=125
counter_driver: unloaded
counter_driver: loaded major=10 minor=125
counter_driver: read value=0
counter_driver: read value=1
counter_driver: set counter=100
counter_driver: read value=100
counter_driver: set counter=0
counter_driver: read value=0
counter_driver: unloaded
/ #
```

Figure 8. Successful rmmod, procfs cleanup, and the kernel log that records load/read/set/unload operations of counter_driver.

Interpreting the kernel log

Actual log line	What it confirms
counter_driver: loaded major=10 minor=125	misc_register succeeded in QEMU
read value=0; read value=1	snapshot and auto-increment behave as expected
set counter=100; set counter=0	ASCII write and reset both work
counter_driver: unloaded	exit() and device deregistration completed

05 | TASK 3

COMPARISON OF CHARACTER / BLOCK / NETWORK DEVICES

The three device classes serve different data units and different kernel paths; the correct framework is selected according to I/O behavior, not merely according to the device name or the physical bus. The table below distinguishes their VFS-facing interfaces, data models, buffering strategies, embedded examples, and suitable usage contexts.

Criterion	Character device	Block device	Network device
Kernel-facing API	file_operations	block_device_operations; blk-mq	net_device_ops; NAPI/socket
Operation unit	Byte stream / record	Addressable block/sector	Packet/frame
VFS / userspace	open/read/write/ioctl on /dev	Filesystem, page cache; /dev	Socket API; not normally read through /dev

Criterion	Character device	Block device	Network device
Buffering	Driver-managed FIFO/buffer	Page cache, bio, request queue	sk_buff, DMA ring, RX/TX queue
Embedded example	UART, char sensor, counter	SD, eMMC, SSD	Ethernet MAC, Wi-Fi, CAN
Best-fit scenario	Streaming data and configuration commands	Random-access sector storage	Frame exchange with a protocol stack

Illustrative design decisions

- A small temperature sensor returns only a few bytes at a time: a character interface or the IIO framework is more appropriate than pretending that it is a block device; the hardware does not expose sector addresses.
- An eMMC partition that stores the root filesystem benefits from the block layer, which provides random access, request merging, and page cache; this differs fundamentally from a byte-stream model.
- A CAN controller transmits identified frames and handles packet-level errors: the network stack (for example SocketCAN) matches that nature far better than arbitrary byte-by-byte read operations.

Task 1 and Task 2 are both character-device families. “Misc” is a simplified registration mechanism for a char device, not a new data class on the same level as block and network.

05 | TASK 3

SYNCHRONIZATION AND MODULE LIFECYCLE

A correct driver does more than create a device node: it must avoid race conditions, handle errors at each initialization step, and reclaim resources properly during unload. The synchronization analysis combines a two-CPU concurrency view with both init/exit lifecycles so that all protected points are visible in one place.

Concurrency and module lifecycle: With an unprotected plain integer, two CPUs can both read 5, compute 6, and write 6, losing one increment. atomic_t prevents a torn or lost individual update, while counter_mutex serializes multi-step transactions and statistics snapshots. On initialization, Task 1 registers its major number, class, and device, whereas Task 2 registers a misc device and creates /proc/counter_info. On unload or initialization failure, Task 1 destroys its device and class before unregistering the character driver; Task 2 removes the procfs entry before calling misc_deregister().

05 | TASK 3

REGISTRATION STRATEGY, ENCODING, AND REAL DEBUG EXPERIENCE

Criterion	Task 1: register_chrdev()	Task 2: misc_register()
Namespace	major 241 chosen by the driver	shared major 10; dynamic minor
Implementation cost	More control; requires class/device setup	Less boilerplate; ideal for a single simple device
Usage scope	Char-driver practice and ioctl design	Standalone counter with one device node

Criterion	Task 1: register_chrdev()	Task 2: misc_register()
Main risk	Major-number collision / init rollback	Access-permission control for the node

Decoding the real commands

The ioctl command is a 32-bit value partitioned into DIR (bits 31–30, 2 bits), SIZE (bits 29–16, 14 bits), TYPE (bits 15–8, 8 bits), and NR (bits 7–0, 8 bits). For this driver, TYPE holds magic value 0x41 (ASCII "A"). The explicit construction and decoding formulas below preserve all four tested command values.

General formula: cmd = (dir << 30) | (size << 16) | (type << 8) | nr. On ARMv7 with Linux 5.15, magic “A” corresponds to TYPE=0x41; sizeof(struct asgn1_stats)=20 bytes. The values below match the student header and the test output exactly.

ASGN1_RESET_BUFFER = _IO('A',1) = (0 << 30) | (0 << 16) | (0x41 << 8) | 1 = 0x00004101

ASGN1_GET_STATS = _IOR('A',2,struct asgn1_stats) = (2 << 30) | (20 << 16) | (0x41 << 8) | 2 = 0x80144102

ASGN1_SET_MODE = _IOW('A',3,int) = (1 << 30) | (4 << 16) | (0x41 << 8) | 3 = 0x40044103

ASGN1_GET_VERSION = _IOR('A',4,char[64]) = (2 << 30) | (64 << 16) | (0x41 << 8) | 4 = 0x80404104

```
_IOC_NR(cmd)   = (cmd >> 0)  & 0xFF
_IOC_TYPE(cmd) = (cmd >> 8)  & 0xFF
_IOC_SIZE(cmd) = (cmd >> 16) & 0x3FFF
_IOC_DIR(cmd)  = (cmd >> 30) & 0x3
```

Magic “A” (0x41) is the TYPE signature of the private ioctl protocol defined by this driver. It helps the driver check the received command and return -ENOTTY when the command does not belong to the expected protocol, thereby reducing the risk of applying an unrelated request to the wrong device state. The check occurs inside the driver that receives the file descriptor; it is not a universal guard against every possible file-descriptor mistake across the entire system.

One proven bug and its fix

During an earlier test phase, the userspace program sent commands built with magic “a” (0x61) while the driver expected magic “A” (0x41); every command became invalid and returned ENOTTY. Using the shared asgn1_ioctl.h header for both the ARM userspace program and the kernel module aligned the ABI and restored the correct command codes 0x4101 / 0x80144102 / 0x40044103 / 0x80404104. After the fix and rebuild, the official log confirms that all four commands pass. This was an ABI mismatch, not a buffer-logic bug.

TASK 4 — QUESTION 1 AND QUESTION 2

Question 1. Module reference counting and unload conditions

Linux maintains a reference count for each module to prevent code from being removed while it is still in use. In a character driver, .owner = THIS_MODULE inside file_operations allows the VFS to hold a reference while a file is open and to release that reference when the file is closed; subsystems or other modules may also hold references through try_module_get()/module_put(). The “Used by” column of lsmod reflects these active module-level references.

If a process still keeps /dev/asgn1 open, a normal safe rmmod will fail because the module is busy; the correct solution is to make the process close the file descriptor, terminate the process, or wait until the activity completes, and only then unload the module. Some configurations support rmmod -f when CONFIG_MODULE_FORCE_UNLOAD is enabled, but that option may leave function pointers and resources referenced after removal, leading to use-after-free errors, Oops messages, or even a panic. Therefore, forced unload is a special debugging option, not a normal operating procedure.

Question 2. Why GET_STATS must use copy_to_user()

On ARMv7-A with the common short-descriptor paging model, when the CPU enters SVC mode and handles ioctl with a process-supplied arg pointer, the MMU translates that virtual address using the page tables of the current process (userspace normally uses TTBR0, subject to TTBCR policy). If the TLB has no matching translation, the MMU performs a page-table walk. A NULL page, an unmapped region, or an invalid PTE causes a Translation Fault; a mapped page without write permission causes a Permission Fault. On ARMv7, a Data Abort reports DFSR (fault status) and DFAR (fault address) so the kernel can diagnose the fault. A demand-paged but otherwise valid page may still be resolved successfully, so not every Data Abort necessarily turns into an Oops.

copy_to_user() is the safe uaccess path: it checks the address range through a mechanism equivalent to access_ok() (not merely a guarantee that a valid PTE exists) and uses copy instructions that are covered by entries in the exception table __ex_table. If the userspace pointer is invalid, the ARM Data Abort handler (the do_DataAbort()/fault-handler path, depending on the kernel) can locate the exception-table fixup, recover execution, and return the number of bytes that could not be copied. The driver checks that return value and then reports -EFAULT. Valid but not-yet-present pages may still be faulted in and copied successfully.

Applied to the Task 1 code

If the driver used memcpy() or directly dereferenced a userspace pointer, it would bypass the address-domain checks and would not guarantee a suitable fixup path. A pointer that accidentally targets kernel memory could overwrite data if permissions and hardware configuration allow it; a pointer that refers to an unmapped region or a read-only page could trigger an unhandled kernel fault/Oops, or even a panic depending on context. This does not mean that every use of memcpy() crashes immediately, but it explains why memcpy() must not replace copy_to_user() for untrusted userspace data. In the real code, GET_STATS uses copy_to_user() and returns -EFAULT on failure.

TASK 4 — QUESTION 3 AND QUESTION 4

Question 3. How is MTD different from the traditional block layer?

A block device presents storage as addressable sectors: upper layers issue read/write requests without needing to know the physical details of each memory cell. In contrast, MTD targets raw flash and does not hide hardware constraints such as eraseblock operations, limited program/erase cycles, bad blocks on NAND, and the need for ECC. One cannot assume that raw NAND behaves like a sector-based disk that allows arbitrary in-place overwrites.

In embedded systems, raw NAND flash connected directly to the SoC (without a Flash Translation Layer) is typically managed by MTD together with UBI/UBIFS or JFFS2 to handle wear leveling and bad blocks. SPI NOR that stores a bootloader or a kernel image may also use MTD when software must erase and write it directly. By contrast, eMMC and SD cards contain an internal controller/FTL and usually appear as block devices that host ext4, FAT, or another filesystem.

Question 4. Why must read() manage *ppos?

The cat utility does not inherently know that “this file contains only one value”. It repeatedly calls read() and writes the received bytes to stdout. As long as read() returns a positive number of bytes, the process continues; when read() returns 0, userspace interprets that as EOF and stops. The VFS maintains the file position (file->f_pos) and passes the corresponding offset to file_operations.read, except in special positioned-I/O cases.

If counter_read() always generated a new number string and always returned a positive byte count without respecting the offset, cat could print forever and never exit, while also incrementing the counter unintentionally. The actual implementation uses simple_read_from_buffer() together with *ppos, plus a per-open snapshot and an initialized flag: the string "N\n" is delivered only once for that file descriptor. On the next read, once the offset is already at the end of the buffer, the function returns 0. Each new cat call performs a fresh open() and therefore sees the next snapshot; that is why two separate cat commands display 0 and then 1.

The relationship between atomicity and file offset

Atomicity protects the counter contents and the update totals; file offset governs the read protocol between the VFS and the application. These are independent concerns: atomic_t cannot replace *ppos, and returning EOF correctly does not by itself eliminate race conditions between CPUs.

07 | SYNTHESIS

EVIDENCE MATRIX, EVALUATION, AND CONCLUSION

Requirement group from the assignment	Evidence in the report / logs
Task 1: four ioctls + mutex	Figures 3–6; test_asgn1_output.txt; dmesg_full.txt
Task 1: load / cleanup	Figures 3 and 6; major 241; “driver unloaded”
Task 2: misc + read/write	Figures 7–8; test_counter_output.txt
Task 2: procfs at 3 checkpoints	Task 2 result table: 0/0/0, 100/2/1, 0/3/2
Task 3: unified diagram	Figure 2 (the only architecture diagram), plus the detailed text on ioctl, counter, synchronization, and command encoding
Task 3: 3 driver classes / design decision	Character / Block / Network table and the selection examples
Task 3: debugged issue	Analysis of the magic mismatch “a”/“A” and the shared header fix
Task 4: four theory answers	The theory pages on references, MMU, MTD, and offset handling

Result evaluation

Task 1 covers both normal and abnormal behavior, including the two modes, statistics retrieval, RESET, version reporting, invalid ioctl handling, and cleanup. Task 2 demonstrates the correct semantics of a misc-interface counter: it increments on successful reads, accepts valid ASCII integer writes, rejects invalid input, and publishes three independent procfs snapshots. Each module was loaded and unloaded successfully on QEMU ARMv7, and both official scripts returned exit code 0.

The main limitation is that the modules simulate kernel-state devices rather than interacting with real hardware; the report does not yet measure performance under heavy load or explore massive parallel read/write scenarios with hundreds of threads. To deploy the design in a real embedded system, further work should include /dev permission hardening, concurrent stress testing, counter-overflow analysis, and rollback validation for partial init failures.

Reference sources

1. ASSIGNMENT01_EmbeddedLinux.docx — the official requirement document containing Tasks 1–4 and the grading rubric.

2. Project source files: task1_ioctl/asgn1_driver.c, asgn1_ioctl.h; task2_misc/counter_driver.c; the Makefiles; and the test scripts.

3. ARMv7 QEMU log set provided by the author: logs/test_asgn1_output.txt; logs/test_counter_output.txt; logs/dmesg_full.txt; logs/build_task1.txt; logs/build_task2.txt.